Deep Learning

Refai Arafat Rossi

October 2026

1 Introduction

[aspectratio=169, 10pt]beamer

Boadilla whale

[utf8]inputenc amsmath, amssymb, amsfonts tikz shapes.geometric, arrows.meta, positioning, calc booktabs multicol

navigation symbols footline[frame number]

[Chapter 6: Deep Feedforward Networks]Deep Feedforward Networks Based on Chapter 6, *Deep Learning* (Ian Goodfellow et al.) Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

height

1. Learning Objectives By the end of this lecture, students should be able to:

- Explain the architecture and fundamental purpose of a **feedforward neural network**.
- Understand why a single linear perceptron cannot learn non-linear functions like **XOR**.
- Explain how introducing a **hidden layer** solves the linear separability constraint.
- Demonstrate why **non-linear activation functions** are mathematically essential.
- Perform step-by-step calculations for individual neurons, layers, and the **ReLU** function.
- Express forward propagation using formal vector and matrix notation $(W^{(l)}, b^{(l)}, h^{(l)})$.
- Walk through the complete 4-step cycle: *Forward Pass* $\rightarrow$ *Loss* $\rightarrow$ *Backpropagation* $\rightarrow$ *Gradient Descent*.

2. Why Do We Need Deep Learning? Fundamental Question Can a machine learn every underlying pattern using a simple linear model?

Problem 1: Predict whether a student passes a course

- Inputs: $x_1 =$ study hours, $\quad x_2 =$ attendance rate
- Target: $y \in \{0, 1\}$ (0 = Fail, 1 = Pass)

A standard linear binary classification model computes:

$$z = w_1 x_1 + w_2 x_2 + b, \quad e.g., z = 2x_1 + 3x_2 - 5$$

$$y = activation(z)$$

Key Limitation Linear models force the decision boundary to be a straight line (or flat hyperplane). Real-world data is rarely linearly separable.

3. What Is a Feedforward Network? Information flows strictly in one direction with no feedback loops:

$$Input(x) \longrightarrow HiddenRepresentation(h) \longrightarrow Output(y)$$

0.5 **Core Characteristics:**

- Information moves strictly left to right.
- No recurrent or loop connections.
- Maps input x to output y via intermediate functions h.

0.5 [x=1.5cm, y=0.9cm, ¿=stealth] [circle, draw, fill=blue!10] (x1) at (0, 1) x_1; [circle, draw, fill=blue!10] (x2) at (0, 0) x_2; [circle, draw, fill=blue!10] (x3) at (0, -1) x_3;
[circle, draw, fill=green!10] (h1) at (1.8, 1) h_1; [circle, draw, fill=green!10] (h2) at (1.8, 0) h_2; [circle, draw, fill=green!10] (h3) at (1.8, -1) h_3;
[circle, draw, fill=red!10] (y) at (3.3, 0) y;
ıin x1, x2, x3 jin h1, h2, h3 [-¿, gray!60] (ı) – (ȷ);
jin h1, h2, h3 [-¿, gray!60] (ȷ) – (y);

4 & 5. The Basic Computational Unit: Neuron Before scaling to deep networks, let's look at a single artificial neuron.

Numerical Example:

- Inputs: $x_1 = 2, \quad x_2 = 3$
- Weights: $w_1 = 0.5, \quad w_2 = 0.2$
- Bias: $b = 1$

x_1	x_2	**XOR** (y)
0	0	0
0	1	1
1	0	1
1	1	0

Step 1: Compute Weighted Sum (z)

$$z = w_1x_1 + w_2x_2 + b = (0.5)(2) + (0.2)(3) + 1 = 1 + 0.6 + 1 = \mathbf{2.6}$$

Step 2: Apply Activation Function (f)

$$a = f(z)$$

Fundamental Neuron Equation

$$a = f(w^T x + b)$$

where $w = w_1$
w_2, $x = x_1$
x_2, $w^T x = \sum_i w_i x_i$, b is bias, and $f(\cdot)$ adds non-linearity.

6 & 7. The Critical Problem: XOR 0.45 **XOR Truth Table:** Output is 1 if and only if the inputs are different.

0.55 **Single Perceptron Decision Boundary:**

$$z = w_1x_1 + w_2x_2 + b = 0$$

This defines a straight line in 2D space:

$$y = \{\, 1\ z > 00z \leq 0$$

Question for Students Can you draw a single straight line that separates the 1s from the 0s?

8 & 9. Visualizing Linear Separability 0.45 [scale=1.8] [-¿, thick] (-0.3,0) – (1.5,0) node[right] x_1; [-¿, thick] (0,-0.3) – (0,1.5) node[above] x_2; [draw, circle, fill=red!40, inner sep=3pt, thick] at (0,0) **0**; [draw, circle, fill=blue!40, inner sep=3pt, thick] at (0,1) **1**; [draw, circle, fill=blue!40, inner sep=3pt, thick] at (1,0) **1**; [draw, circle, fill=red!40, inner sep=3pt, thick] at (1,1) **0**;
[dashed, red, thick] (-0.2, 0.6) – (1.4, -0.2); [dashed, red, thick] (-0.2, 1.4) – (1.4, 0.2);

XOR: Not Linearly Separable

0.55 **Comparison of Boolean Functions:**

- **AND:** Linearly separable (∃ separating hyperplanes).
- **OR:** Linearly separable (∃ separating hyperplanes).

- **XOR: NOT linearly separable**.

Takeaway

$$A single perceptron cannot learn XOR.$$

Linear models fail on simple non-linear patterns.

10, 11 & 12. Solution: Add a Hidden Layer We can decompose XOR using basic logic gates:

$$x_1 \oplus x_2 = (x_1 OR x_2) AND NOT (x_1 AND x_2)$$

0.5 **Constructing Hidden Representations:**

- Neuron $h_1 = x_1 OR x_2$
- Neuron $h_2 = x_1 AND x_2$
- Output $y = h_1 AND NOT h_2$

The hidden layer transforms the original input space into a new feature space (h_1, h_2) where the problem becomes linearly separable!

0.5 [x=1.8cm, y=1.2cm, ¿=stealth] [circle, draw, fill=blue!10] (x1) at (0, 0.8) x_1; [circle, draw, fill=blue!10] (x2) at (0, -0.8) x_2; [circle, draw, fill=green!10, align=center] (h1) at (1.5, 0.8) h_1 (OR); [circle, draw, fill=green!10, align=center] (h2) at (1.5, -0.8) h_2 (AND); [circle, draw, fill=red!10] (y) at (3, 0) y; [-¿] (x1) – (h1); [-¿] (x1) – (h2); [-¿] (x2) – (h1); [-¿] (x2) – (h2); [-¿] (h1) – (y); [-¿] (h2) – (y);

Representation Learning:

$$x \longrightarrow h \longrightarrow y$$

14 & 15. Why Non-Linear Activation Functions Are Mandatory What happens if we stack multiple linear layers without activation functions?

Mathematical Proof: h = $W_1 x + b_1$ $(HiddenLayer)$ $y = W_2 h + b_2$ $(OutputLayer)$ Substitute h into y:

$$y = W_2(W_1 x + b_1) + b_2 = (\mathbf{W_2 W_1})\mathbf{x} + (\mathbf{W_2 b_1} + \mathbf{b_2})$$

Let $W' = W_2 W_1$ and $b' = W_2 b_1 + b_2$:

$$y = W'x + b' \quad \Longleftarrow \textbf{Still a purely linear function!}$$

Crucial Theorem

$$Many Linear Layers \equiv One Single Linear Layer$$

Non-linear activation functions (f) are mandatory to enable universal function approximation:

$$h = f(Wx + b)$$

16, 17, 18 & 19. Rectified Linear Unit (ReLU) 0.5 **Definition:**

$$ReLU(x) = \max(0, x) = \{\, 0\ x < 0xx \geq 0$$

Quick Examples:

- $ReLU(5) = 5$
- $ReLU(-3) = 0$
- $ReLU(0) = 0$
- $ReLU(-2.5) = 0$

Why ReLU dominates Modern Deep Learning:

1. Computationally efficient (simple thresholding).
2. Introduces necessary non-linearity.
3. Avoids gradient saturation for positive inputs.

0.5 [scale=0.9] [-¿, thick] (-3,0) – (3,0) node[right] x; [-¿, thick] (0,-0.5) – (0,3) node[above] y;
[ultra thick, blue] (-3,0) – (0,0); [ultra thick, blue] (0,0) – (2.5,2.5) node[right] $y = \max(0, x)$;
[blue, below] at (-1.5,0) $y = 0$; [blue, right] at (1.2,1) $y = x$; [fill=blue] (0,0) circle (2pt);

21 & 22. ReLU Numerical Problems Problem 1 Given $x_1 = 2, x_2 = 3$, weights $w_1 = 0.5, w_2 = -1$, and bias $b = 1$. Calculate the neuron output using ReLU. **Solution:**

1. Compute z:

$$z = (0.5)(2) + (-1)(3) + 1 = 1 - 3 + 1 = -1$$

2. Apply activation:

$$a = ReLU(-1) = \mathbf{0}$$

Problem 2 Given $x_1 = 4, x_2 = 2$, weights $w_1 = 0.5, w_2 = 0.5$, and bias $b = -1$. **Solution:**

1. Compute z:

$$z = (0.5)(4) + (0.5)(2) - 1 = 2 + 1 - 1 = 2$$

2. Apply activation:

$$a = ReLU(2) = \mathbf{2}$$

23 & 24. Matrix Vector Computation for a Layer For multiple neurons in a layer, we express operations in matrix form:

$$h = f(Wx + b)$$

Classroom Exercise: Given input $x = 2$
1, weight matrix $W = 12$
$-$ 11, and bias $b = 1$
$-$ 2. Compute h using ReLU activation.
Step-by-Step Calculation:

1. Compute Matrix Product Wx:

$$Wx = 12 - 1121 = (1)(2) + (2)(1)(-1)(2) + (1)(1) = 4 - 1$$

2. Add Bias Vector b:

$$z = Wx + b = 4 - 1 + 1 - 2 = 5 - 3$$

3. Apply Element-wise ReLU:

$$h = ReLU\,(5 - 3) = ReLU(5)ReLU(-3) = \mathbf{50}$$

25, 26, 27 & 28. Deep Feedforward Architecture & Notation When we stack multiple hidden layers, we create a **Deep Feedforward Network** or **Multilayer Perceptron (MLP)**.

Notation Standard Superscripts in parentheses (l) denote the **layer index**, NOT exponentiation!

- $W^{(l)}$: Weight matrix for layer l
- $b^{(l)}$: Bias vector for layer l
- $h^{(l)}$: Output vector (hidden activation) of layer l

Full Mathematical Equations for a 2-Hidden-Layer Network: **Layer 1:** $z^{(1)} = W^{(1)}x + b^{(1)}, h^{(1)} = f(z^{(1)})$
Layer 2: $z^{(2)} = W^{(2)}h^{(1)} + b^{(2)}, h^{(2)} = f(z^{(2)})$
Output: $z^{(3)} = W^{(3)}h^{(2)} + b^{(3)}, \hat{y} = g(z^{(3)})$ where $g(\cdot)$ is the output layer activation (e.g., Sigmoid, Softmax, or Linear).

29 & 30. Complete Forward Propagation Numerical Example **Given Architecture:** 2 Inputs $\rightarrow$ 2 Hidden Neurons (ReLU) $\rightarrow$ 1 Output (Sigmoid σ).
Inputs & Parameters:

$$x = 12, \quad W^{(1)} = 112 - 1, \quad b^{(1)} = 01, \quad W^{(2)} = 2 - 1, \quad b^{(2)} = 0$$

Step 1: Compute Hidden Layer activations ($h^{(1)}$)

$$z^{(1)} = W^{(1)}x + b^{(1)} = (1)(1) + (1)(2) + 0(2)(1) + (-1)(2) + 1 = 31$$

$$h^{(1)} = ReLU\,(31) = 31$$

Step 2: Compute Final Output activation ($\hat{y}$)

$$z^{(2)} = W^{(2)}h^{(1)} + b^{(2)} = 2 - 131 + 0 = (2)(3) + (-1)(1) = 5$$

$$\hat{y} = \sigma(z^{(2)}) = \frac{1}{1+e^{-5}} \approx \mathbf{0.993} \quad (99.3\%predictedprobability)$$

31 & 32. What Is Learning? Loss Functions

$$\text{x} \longrightarrow W^{(1)}x + b^{(1)} \longrightarrow ReLU \longrightarrow W^{(2)}h + b^{(2)} \longrightarrow Output\,(\hat{y})$$

How do we find weights W and biases b?

- Parameters are initialized to small random values.
- Predictions ($\hat{y}$) are compared against actual target labels (y).
- Discrepancy is measured using a **Loss Function** $L(y, \hat{y})$.

Simple Numerical Example (Mean Squared Error):

- Actual value $y = 1.0$, Prediction $\hat{y} = 0.7$
- Squared Error Loss:

$$L = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1.0 - 0.7)^2 = \frac{1}{2}(0.09) = \mathbf{0.045}$$

Goal of Training: Adjust parameters $\theta = \{W, b\}$ to minimize L.

34. Parameters vs. Hyperparameters It is essential for Master's students to distinguish between these two concepts:

35 & 36. Gradient Descent Optimization To minimize loss L, we update weights in the direction opposite to the gradient:

$$w_{new} = w_{old} - \eta\frac{\partial L}{\partial w}$$

where $\eta > 0$ is the **learning rate**.

0.5 **Intuition:**

- Derivative $\frac{\partial L}{\partial w} > 0$ Decrease w.
- Derivative $\frac{\partial L}{\partial w} < 0$ Increase w.

Parameters
Learned automatically during training via Optimization.

- Weights ($W^{(l)}$)
- Biases ($b^{(l)}$)

Hyperparameters
Chosen manually by the model designer before training.

- Learning rate (η)
- Number of hidden layers
- Number of neurons per layer
- Activation functions (ReLU, Sigmoid)
- Batch size & Epochs

Property	**Sigmoid**	**ReLU**
Formula	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\max(0, x)$
Output Range	$(0, 1)$	$[0, \infty)$
Computation Cost	High (exponential function)	Extremely simple
Gradient Issue	Vanishing gradient problem	No saturation for $x > 0$
Primary Use Case	**Binary Output Layer**	**Hidden Layers**

0.5 Worked Numerical Problem Suppose initial weight $w = 2.0$, gradient $\frac{\partial L}{\partial w} = 4.0$, and learning rate $\eta = 0.1$.

Compute updated weight:

$$w_{new} = 2.0 - (0.1)(4.0) = 2.0 - 0.4 = \mathbf{1.6}$$

37, 38 & 39. Backpropagation & Derivatives **The 4-Step Deep Learning Training Loop:**

$$ForwardPass \longrightarrow ComputeLoss \longrightarrow Backpropagation \longrightarrow WeightUpdate$$

Role of Backpropagation: Uses the calculus **chain rule** to compute partial gradients $\frac{\partial L}{\partial W^{(l)}}$ layer by layer from output to input.

ReLU Derivative Function:

$$ReLU'(z) = \{\, 0\ z < 01z > 0$$

Quick Derivative Check

- If $z = -5 ReLU'(-5) = \mathbf{0}$ (Gradients are blocked / Zero flow).
- If $z = 4 ReLU'(4) = \mathbf{1}$ (Gradients pass through unchanged).

40 & 41. Comparison: Activation Functions

Architecture Selection Summary

- **Hidden Layers:** Use **ReLU** (or variants like LeakyReLU).
- **Binary Output Layer:** Use **Sigmoid** (outputs probability in $[0, 1]$).
- **Multiclass Output Layer:** Use **Softmax** (outputs probability distribution across classes).

42 & 43. Classroom Exercises (Interactive) Exercise 1: Single Neuron Calculation Given $x = [3, 2]^T, w = [2, -1]^T, b = -2$. Calculate $ReLU(w^T x + b)$.

Solution: $z = (2)(3) + (-1)(2) - 2 = 6 - 2 - 2 = 2ReLU(2) = \mathbf{2}$.

Exercise 2: Matrix Layer Calculation Given $x = 1$
$3, W = 2 - 1$
$12, b = 1$
-1. Compute $h = ReLU(Wx + b)$.

Solution:

$$Wx = 2(1) - 1(3)1(1) + 2(3) = -17z = -17 + 1 - 1 = 06h = \mathbf{06}$$

44, 45 & 46. Conceptual Exercises **Exercise 3: Problem Identification**

- Which of the following problems **cannot** be solved by a single perceptron? [A.]

  1. AND B. OR **C. XOR** D. Linear Regression

- *Reason:* XOR is not linearly separable.

Exercise 4: Quick ReLU Evaluation

- $ReLU(-7) = \mathbf{0}, \quad ReLU(-0.5) = \mathbf{0}, \quad ReLU(0) = \mathbf{0}$
- $ReLU(3) = \mathbf{3}, \quad ReLU(10.5) = \mathbf{10.5}$

Exercise 5: Conceptual Thinking

- *Question:* If a network has 10 layers but NO non-linear activation functions, is it a deep non-linear model?
- *Answer:* **No.** Stacking linear transformations yields another linear transformation ($W_{10} \cdots W_1 x = W' x$).

47. Recommended Teaching Flow (3–4 Hour Breakdown) 0.5 **Part A: Motivation & Perceptron (30m)**

- Linear models, weights, bias, neuron formula.

Part B: The XOR Benchmark (40m)

- Truth tables, linear separability, failure of single perceptron, hidden layer intuition.

Part C: Activation Functions (35m)

- Why non-linearity matters, ReLU definition, graphs, numerical examples.

0.5 **Part D: Deep Feedforward Networks (45m)**

- Multi-layer notation ($W^{(l)}$), matrix forward propagation.

Part E: Learning & Optimization (40m)

- Loss functions, gradient descent updates, backprop intuition.

Part F: Student Exercises (30–40m)

- Hands-on matrix, neuron, and gradient math problems.

48. The 5 Core Equations Every Student Must Master

1. **Single Neuron Computation:**

$$z = w^T x + b$$

2. **Neuron Output with Activation:**

$$a = f(z)$$

3. **Layer Representation Matrix Form:**

$$h^{(l)} = f\left(W^{(l)} h^{(l-1)} + b^{(l)}\right)$$

4. **Rectified Linear Unit (ReLU):**

$$ReLU(x) = \max(0, x)$$

5. **Gradient Descent Parameter Update:**

$$\theta_{new} = \theta_{old} - \eta \nabla_\theta L$$

49. Final Summary **The Progression of Feedforward Neural Networks:**

Linear Models $\longrightarrow$ Single Perceptron $\longrightarrow$ Failure on XOR

$\Downarrow$

Add Hidden Layer $\longrightarrow$ Non-linear Activation (ReLU) $\longrightarrow$ Deep Feedforward Network

$\Downarrow$

Forward Propagation $\longrightarrow$ Loss Calculation $\longrightarrow$ Backpropagation $\longrightarrow$ Gradient Descent

Core Conceptual Takeaway

$Deep Learning learns hierarchical representations through chained non-linear transformations.$